

Path 2: Cyber Security 101 > Module 4: Command Line > Room 2: Windows PowerShell

<https://tryhackme.com/room/windowspowershell>

Windows PowerShell:

Discover the "Power" in PowerShell and learn the basics.

>> Task 1: Introduction

♦ Learning Objectives:

This is the second room in the **Command Line** module. It is an introductory room to PowerShell, the second—only historically—command-line utility built for the Windows operating system.

- Learn what PowerShell is and its capabilities.
- Understand the basic structure of PowerShell's language.
- Learn and run some basic PowerShell commands.
- Understand PowerShell's many applications in the cyber security industry.

Room Prerequisites:

Before approaching this room, it's recommended that you have understood the concepts in the **Windows and AD Fundamentals** module and the **Windows Command Line** room.

>> Task 2: What Is PowerShell

From the official Microsoft page (<https://learn.microsoft.com/en-us/powershell/scripting/overview?view=powershell-7.4>): "PowerShell is a cross-platform task automation solution made up of a command-line shell, a scripting language, and a configuration management framework."

PowerShell is a powerful tool from Microsoft designed for task automation and configuration management. It combines a command-line interface and a scripting language built on the .NET framework. Unlike older text-based command-line tools, PowerShell is "object-oriented", which means it can handle complex data types and interact with system components more effectively. Initially exclusive to Windows, PowerShell has lately expanded to support macOS and Linux, making it a versatile option for IT professionals across different operating systems.

♦ The Power in PowerShell

To fully grasp the power of PowerShell, we first need to understand what an object is in this context.

In programming, an object represents an item with properties (characteristics) and methods (actions). For example, a **car** object might have properties like **Color**, **Model**, and **FuelLevel**, and methods like **Drive()**, **HonkHorn()**, and **Refuel()**.

Similarly, in PowerShell, objects are fundamental units that encapsulate data and functionality, making it easier to manage and manipulate information. An object in PowerShell can contain file names, usernames or sizes as data (**properties**), and carry functions (**methods**) such as copying a file or stopping a process.

Q1) What do we call the advance approach to develop PowerShell?

Answer: Object-Oriented

>> Task 3: PowerShell Basics

♦ Launching PowerShell:

PowerShell can be launched in several ways, depending on your needs and environment. If you are working on a Windows system from the graphical interface (GUI), these are some of the possible ways to launch it:

- **Start Menu:** Type **powershell** in the Windows Start Menu search bar, then click on **Windows PowerShell** or **PowerShell** from the results.
- **Run Dialog:** Press **Win + R** to open the **Run** dialog, type **powershell**, and hit **Enter**.
- **File Explorer:** Navigate to any folder, then type **powershell** in the address bar, and press **Enter**. This opens PowerShell in that specific directory.
- **Task Manager:** Open the Task Manager, go to **File > Run new task**, type **powershell**, and press **Enter**.

Alternatively, PowerShell can be launched from a **Command Prompt (cmd.exe)** by typing **powershell**, and pressing **Enter**.

♦ Basic Syntax: Verb-Noun

As previously mentioned, PowerShell commands are known as **cmdlets** (pronounced **command-lets**). They are much more powerful than the traditional Windows commands and allow for more advanced data manipulation.

Cmdlets follow a consistent **Verb-Noun** naming convention. This structure makes it easy to understand what each cmdlet does. The **Verb** describes the action, and the **Noun** specifies the object on which action is performed. For example:

Get-Content: Retrieves (gets) the content of a file and displays it in the console.

Set-Location: Changes (sets) the current working directory.

◆ Basic Cmdlets:

To list all available cmdlets, functions, aliases, and scripts that can be executed in the current PowerShell session, we can use **Get-Command**. It's an essential tool for discovering what commands one can use.

For each **CommandInfo** object retrieved by the cmdlet, some essential information (properties) is displayed on the console. It's possible to filter the list of commands based on displayed property values. For example, if we want to display only the available commands of type "function", we can use **-CommandType "Function"**. For Example: **Get-Command -CommandType "Function"**

Another essential cmdlet to keep in our tool belt is **Get-Help**: it provides detailed information about cmdlets, including usage, parameters, and examples. It's the go-to cmdlet for learning how to use PowerShell commands.

Command: **Get-Help Get-Date**

REMARKS:

To see the examples, type: **get-help Get-Date -examples**.

For more information, type: **get-help Get-Date -detailed**.

For technical information, type: **get-help Get-Date -full**.

For online help, type: **get-help Get-Date -online**.

To make the transition easier for IT professionals, PowerShell includes aliases—which are shortcuts or alternative names for cmdlets—for many traditional Windows commands. Indispensable for users already familiar with other command-line tools, **Get-Alias** lists all aliases available. For example, **dir** is an alias for **Get-ChildItem**, and **cd** is an alias for **Set-Location**.

◆ Where to Find and Download Cmdlets:

Another powerful feature of PowerShell is the possibility of extending its functionality by downloading additional cmdlets from online repositories.

To search for modules (collections of cmdlets) in online repositories like the PowerShell Gallery, we can use **Find-Module**. Sometimes, if we don't know the exact name of the module, it can be useful to search for modules with a similar name. We can achieve this by filtering the **Name** property and appending a wildcard (*) to the module's partial name, using the following standard PowerShell syntax: **Cmdlet -Property "pattern*"**.

- Example: `Find-Module -Name "PowerShell*"`

Once identified, the modules can be downloaded and installed from the repository with `Install-Module`, making new cmdlets contained in the module available for use.

- Example: `Install-Module -Name "PowerShellGet"`

Q1) How would you retrieve a list of commands that **start with** the verb `Remove`?
[for the sake of this question, avoid the use of quotes (" or ') in your answer]

Answer: `Get-Command -Name Remove*`

Q2) What cmdlet has its traditional counterpart `echo` as an alias?

Answer: `Write-Output`

Hint: `Get-Alias`

Hint: `Get-Command -Name "echo"`

Hint: `Get-Command -Name echo`

Q3) What is the command to retrieve some example usage for the cmdlet `New-LocalUser`?

Answer: `Get-Help New-LocalUser -examples`

>> Task 4: Navigating the File System and Working with Files

Similar to the **dir** command in Command Prompt (or **ls** in Unix-like systems), **Get-ChildItem** lists the files and directories in a location specified with the **-Path** parameter. It can be used to explore directories and view their contents. If no **Path** is specified, the cmdlet will display the content of the current working directory.

Command: **Get-ChildItem**

Command: **Get-ChildItem -Path <Path>**

To navigate to a different directory, we can use the **Set-Location** cmdlet. It changes the current directory, bringing us to the specified path, akin to the **cd** command in Command Prompt.

Command: **Set-Location -Path <Path>**

Command: **Set-Location -Path ".\Documents\"**

Command: **Set-Location -Path .\Documents**

Command: **Set-Location -Path Documents**

While the traditional Windows CLI uses separate commands to create and manage different items like directories and files, PowerShell simplifies this process by providing a single set of cmdlets to handle the creation and management of both files and directories.

Command: **New-Item -Path ".\captain-cabin\captain-wardrobe" -ItemType "Directory"**

Command:

New-Item -Path ".\captain-cabin\captain-wardrobe\captain-boots.txt" -ItemType "File"

Similarly, the **Remove-Item** cmdlet removes both directories and files, whereas in Windows CLI we have separate commands **rmdir** and **del**.

Command: **Remove-Item -Path ".\captain-cabin\captain-wardrobe\captain-boots.txt"**
(Deleting File)

Command: **Remove-Item -Path ".\captain-cabin\captain-wardrobe"**
(Deleting Directory)

We can copy or move files and directories alike, using respectively **Copy-Item** (equivalent to **copy**) and **Move-Item** (equivalent to **move**).

Command: **Copy-Item -Path .\captain-cabin\captain-hat.txt -Destination .\captain-cabin\captain-hat2.txt** (Copying File)

Command: **Move-Item -Path .\captain-cabin\captain-wardrobe -Destination C:\Users\captain** (Moving Directory)

Finally, to read and display the contents of a file, we can use the **Get-Content** cmdlet, which works similarly to the **type** command in Command Prompt (or **cat** in Unix-like systems).

Command: **Get-Content -Path <Path>**

Command: **Get-Content -Path ".\captain-hat.txt"**

Q1) What cmdlet can you use instead of the traditional Windows command **type**?

Answer: **Get-Content**

Q2) What PowerShell command would you use to display the content of the "C:\Users" directory? [for the sake of this question, avoid the use of quotes (" or ') in your answer]

Answer: **Get-ChildItem -Path C:\Users**

Q3) How many items are displayed by the command described in the previous question?

Answer: 4

>> Task 5: Piping, Filtering, and Sorting Data

Piping is a technique used in command-line environments that allows the output of one command to be used as the input for another. This creates a sequence of operations where the data flows from one command to the next. Represented by the **|** symbol, piping is widely used in the Windows CLI, as introduced earlier in this module, as well as in Unix-based shells.

In PowerShell, piping is even more powerful because it passes **objects** rather than just text. These objects carry not only the data but also the properties and methods that describe and interact with the data.

For example, if you want to get a list of files in a directory and then sort them by size, you could use the following command in PowerShell:

Command: `Get-ChildItem | Sort-Object Length`

Here, `Get-ChildItem` retrieves the files (as objects), and the pipe (**|**) sends those file objects to `Sort-Object`, which then sorts them by their **Length** (size) property. This object-based approach allows for more detailed and flexible command sequences.

In the example above, we have leveraged the `Sort-Object` cmdlet to sort objects based on specified properties. Beyond sorting, PowerShell provides a set of cmdlets that, when combined with piping, allow for advanced data manipulation and analysis.

To filter objects based on specified conditions, returning only those that meet the criteria, we can use the `Where-Object` cmdlet. For instance, to list only **.txt** files in a directory, we can use:

Command: `Get-ChildItem | Where-Object -Property "Extension" -eq ".txt"`

Here, `Where-Object` filters the files by their **Extension** property, ensuring that only files with extension equal (**-eq**) to **.txt** are listed.

The operator **-eq** (i.e. "**equal to**") is part of a set of **comparison operators** that are shared with other scripting languages (e.g. Bash, Python). To show the potentiality of the PowerShell's filtering, we have selected some of the most useful operators from that list:

-ne: "**not equal**". This operator can be used to exclude objects from the results based on specified criteria.

-gt: "**greater than**". This operator will filter only objects which exceed a specified value. It is important to note that this is a strict comparison, meaning that objects that are equal to the specified value will be excluded from the results.

-ge: "greater than or equal to". This is the non-strict version of the previous operator. A combination of **-gt** and **-eq**.

-lt: "less than". Like its counterpart, "greater than", this is a strict operator. It will include only objects which are strictly below a certain value.

-le: "less than or equal to". Just like its counterpart **-ge**, this is the non-strict version of the previous operator. A combination of **-lt** and **-eq**.

Below, another example shows that objects can also be filtered by selecting properties that match (**-like**) a specified pattern:

Command: **Get-ChildItem | Where-Object -Property "Name" -like "ship*"**

The next filtering cmdlet, **Select-Object**, is used to select specific properties from objects or limit the number of objects returned. It's useful for refining the output to show only the details one needs.

Command: **Get-ChildItem | Select-Object Name,Length**

Exercise: The cmdlets pipeline can be extended by adding more commands, as the feature isn't limited to just piping between two cmdlets. As an exercise, try and build a pipeline of cmdlets to sort and filter the output with the goal of displaying the largest file in the **C:\Users\captain\Documents\captain-cabin** directory.

Command: **Get-ChildItem | Sort-Object Length -Descending | Select-Object -First 1**

The last in this set of filtering cmdlets is **Select-String**. This cmdlet searches for text patterns within files, similar to **grep** in Unix-based systems or **findstr** in Windows Command Prompt. It's commonly used for finding specific content within log files or documents.

Command: **Select-String -Path ".\captain-hat.txt" -Pattern "hat"**

The **Select-String** cmdlet fully supports the use of regular expressions (**regex** [<https://learn.microsoft.com/en-us/dotnet/standard/base-types/regular-expressions>]).

This advanced feature allows for complex pattern matching within files, making it a powerful tool for searching and analysing text data.

Q1) How would you retrieve the items in the current directory with size greater than 100? [for the sake of this question, avoid the use of quotes (" or ') in your answer]

Answer: **Get-ChildItem | Where-Object -Property Length -gt 100**

>> Task 6: System and Network Information

PowerShell was created to address a growing need for a powerful automation and management tool to help system administrators and IT professionals. As such, it offers a range of cmdlets that allow the retrieval of detailed information about system configuration and network settings.

- ♦ The **Get-ComputerInfo** cmdlet retrieves comprehensive system information, including operating system information, hardware specifications, BIOS details, and more. It provides a snapshot of the entire system configuration in a single command. Its traditional counterpart "systeminfo" retrieves only a small set of the same details.

Command: **Get-ComputerInfo**

Command: **systeminfo**

- ♦ Essential for managing user accounts and understanding the machine's security configuration, **Get-LocalUser** lists all the local user accounts on the system. The default output displays, for each user, username, account status, and description.

Command: **Get-LocalUser**

- ♦ Similar to the traditional **ipconfig** command, the following two cmdlets can be used to retrieve detailed information about the system's network configuration.

Get-NetIPConfiguration provides detailed information about the network interfaces on the system, including IP addresses, DNS servers, and gateway configurations.

Command: **Get-NetIPConfiguration**

- ♦ In case we need specific details about the IP addresses assigned to the network interfaces, the **Get-NetIPAddress** cmdlet will show details for all IP addresses configured on the system, including those that are not currently active.

Command: **Get-NetIPAddress**

Q1) Other than your current user and the default "Administrator" account, what other user is enabled on the target machine?

Answer: p1r4t3

Hint: `Get-localUser`

Q2) This lad has hidden his account among the others with no regard for our beloved captain! What is the motto he has so bluntly put as his account's description?

Answer: A merry life and a short one.

Q3) Now a small challenge to put it all together. This shady lad that we just found hidden among the local users has his own home folder in the "C:\Users" directory. Can you navigate the filesystem and find the hidden treasure inside this pirate's home?

Answer: THM{p34rIlnAsh3II}

>> Task 7: Real-Time System Analysis

To gather more advanced system information, especially concerning dynamic aspects like running processes, services, and active network connections, we can leverage a set of cmdlets that go beyond static machine details.

- ♦ Command: **Get-Process**

Get-Process provides a detailed view of all currently running processes, including CPU and memory usage, making it a powerful tool for monitoring and troubleshooting.

- ♦ Command: **Get-Service**

Similarly, **Get-Service** allows the retrieval of information about the status of services on the machine, such as which services are running, stopped, or paused. It is used extensively in troubleshooting by system administrators, but also by forensics analysts hunting for anomalous services installed on the system.

- ♦ Command: **Get-NetTCPConnection**

To monitor active network connections, **Get-NetTCPConnection** displays current TCP connections, giving insights into both local and remote endpoints. This cmdlet is particularly handy during an incident response or malware analysis task, as it can uncover hidden backdoors or established connections towards an attacker-controlled server.

- ♦ Command: **Get-FileHash**

Additionally, we are going to mention **Get-FileHash** as a useful cmdlet for generating file hashes, which is particularly valuable in incident response, threat hunting, and malware analysis, as it helps verify file integrity and detect potential tampering.

Syntax: **Get-FileHash -Path <File/FilePath>**

- ♦ You can also view the Alternate Data Streams (ADS) attached to a file through PowerShell: (If you are not sure what ADS is, we have covered it in detail in the **Windows Fundamentals 1** room)

You can see two streams that are attached to the file "C:\House\house_log.txt":

1) **:\$DATA** is the default data stream of every NTFS file. It contains the normal file contents and is not an ADS.

2) **housinginfo** is the Alternate Data Stream (ADS) added to this file. It appears as house_log.txt:housinginfo, which means an extra hidden stream named housinginfo is attached to this file.

Q1) In the previous task, you found a marvellous treasure carefully hidden in the target machine. What is the hash of the file that contains it? [File: big-treasure.txt]

Answer:

71FC5EC11C2497A32F8F08E61399687D90ABE6E204D2964DF589543A613F3E08

Hint: `Get-FileHash -Path <File/FilePath>`

Q2) What property retrieved by default by `Get-NetTCPConnection` contains information about the process that has started the connection?

Answer: OwningProcess

Hint: `Get-NetTCPConnection`

Q3) It's time for another small challenge. Some vital service has been installed on this pirate ship to guarantee that the captain can always navigate safely. But something isn't working as expected, and the captain wonders why. Investigating, they find out the truth, at last: the service has been tampered with! The shady lad from before has modified the service `DisplayName` to reflect his very own motto, the same that he put in his user description.

With this information and the PowerShell knowledge you have built so far, can you find the service name?

Answer: p1r4t3-s-compass

Hint: `Get-Service | Where-Object DisplayName -like "*merry*"`

>> Task 8: Scripting

Scripting is the process of writing and executing a series of commands contained in a text file, known as a script, to automate tasks that one would generally perform manually in a shell, like PowerShell.

Simply speaking, scripting is like giving a computer a to-do list, where each line in the script is a task that the computer will carry out automatically. This saves time, reduces the chance of errors, and allows to perform tasks that are too complex or tedious to do manually. As you learn more about shells and scripting, you'll discover that scripts can be powerful tools for managing systems, processing data, and much more.

- ♦ Learning scripting with PowerShell goes beyond the scope of this room.

Nonetheless, we must understand that its power makes it a crucial skill across all cyber security roles.

- For **blue team** professionals such as incident responders, malware analysts, and threat hunters, PowerShell scripts can automate many different tasks, including log analysis, detecting anomalies, and extracting indicators of compromise (IOCs).

These scripts can also be used to reverse-engineer malicious code (malware) or automate the scanning of systems for signs of intrusion.

- For the **red team**, including penetration testers and ethical hackers, PowerShell scripts can automate tasks like system enumeration, executing remote commands, and crafting obfuscated scripts to bypass defences. Its deep integration with all types of systems makes it a powerful tool for simulating attacks and testing systems' resilience against real-world threats.

- Staying in the context of cyber security, **system administrators** benefit from PowerShell scripting for automating integrity checks, managing system configurations, and securing networks, especially in remote or large-scale environments. PowerShell scripts can be designed to enforce security policies, monitor systems health, and respond automatically to security incidents, thus enhancing the overall security posture.

Whether used defensively or offensively, PowerShell scripting is an essential capability in the cyber security toolkit.

♦ Command: **Invoke-Command**

Before concluding this task about scripting, we can't go without mentioning the **Invoke-Command** cmdlet.

Invoke-Command is essential for executing commands on remote systems, making it fundamental for system administrators, security engineers and penetration testers. **Invoke-Command** enables efficient remote management and—combining it with scripting—automation of tasks across multiple machines. It can also be used to execute payloads or commands on target systems during an engagement by penetration testers—or attackers alike.

Let us discover some example usage for this powerful cmdlet by consulting the **Get-Help "examples"** page:

Command: **Get-Help Invoke-Command -examples** (Run in PowerShell to see result)

- The first example shows how the cmdlet can be very easily combined with any custom script to automate tasks on remote computers.

- The second example demonstrates that we don't need to know how to script to benefit from the power of **Invoke-Command**. In fact, by appending the **-ScriptBlock { ... }** parameter to the cmdlet's syntax, we can execute any command (or sequence of commands) on the remote computer. The result would be the same as if we were typing the commands in a local PowerShell session on the remote computer itself.

Q1) What is the syntax to execute the command **Get-Service** on a remote computer named "RoyalFortune"? Assume you don't need to provide credentials to establish the connection. [for the sake of this question, avoid the use of quotes (" or ') in your answer]

Answer:

Invoke-Command -ComputerName RoyalFortune -ScriptBlock { Get-Service }

Hint: **Get-Help Invoke-Command -examples** (To see syntax)